

LLM Logic Task 5: Linear Temporal Logic Tutorial

Linear Temporal logic (LTL) is a highly regarded language for formal verification of safety critical systems. However they are also very unintuitive so this tutorial will serve as a guide to using Linear Temporal logic for reasoning.

The syntax of LTL consists of propositional variables, logical operators, and temporal modal operators. Most may be familiar with propositional variables and logical operators like $\neg$ and $\vee$ but temporal modal operators may confuse some. Some of these operators include G, F, X, and U. The operators represent the following:

- G p: always true if p always holds along the path
- F p: true if p holds at some state along the path
- X p: true on paths in state x if p holds in next state y
- p U q: NOT p union q, but p Until q
 - True along a path starting at s if q is true in some states reachable from s
 - p is true in all states from s until q holds

As stated previously, LTL is primarily used to verify systems and an example of this is checking if a system will eventually reach a state where some condition p is true. This could be represented as F p, meaning that at some point/state, the condition p will be true.

The research paper “What’s in a Name? Linear Temporal Logic Literally Represents Time Lines” tackles the issue of LTL being unintuitive by representing LTL through timelines by translating them to Buchi automata and then to regex, simplifying the expressions before visualization as timelines. This paper is one example of LTL’s utility and potential through this advancement, showing how it can be further improved.

Everything discussed so far in this tutorial are properties expressed over a single computation path/run. This makes up LTL but there are various other forms of Temporal Logic such as Computation Tree logic that this tutorial will not dive into.

Github: